

AI for AI infrastructure: speed is the competitive edge

Perspective

- **Frontier models are more than compute workloads. Using them well strengthens the AI stack and accelerates data, model, software and hardware iteration.** [Forecast: labs will use their own frontier models more widely to raise internal productivity.]
- **The future is uncertain. Rapidly delivering infrastructure that best fits evolving workloads is a clear competitive advantage.**
 - i. **Autonomous coding agents raise productivity: better code extracts more performance from the same hardware.** [Forecast: broad coverage by end-2025, approaching top human experts on most code in 2026.]
 - i. **Models accelerate chip, circuit and system design.** [Forecast: Google and NVIDIA will expand AI hardware design, potentially cutting chip design cycles to six months.]
 - i. **Synthetic open-ended environments and automated algorithm search could enable rapid model self-improvement.** [Forecast: benchmarks will shift to real tasks. Self-evolution remains out of reach until creativity, coding, long-term memory and feedback cross a critical threshold.]
 - i. **General-purpose hardware and software must support AI development.** [Forecast: more progress toward models as operating systems and early ecosystems in 2025.]

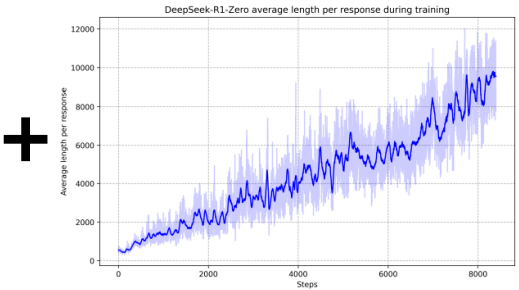
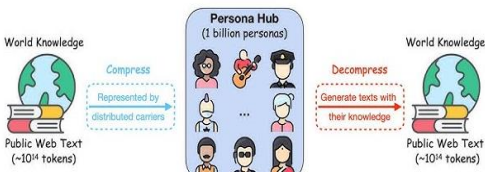
AI accelerates the technology stack and compute infrastructure

New scaling laws: synthetic data and RL shorten model cycles

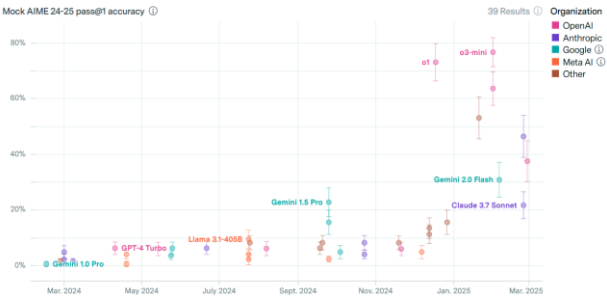
sakana.ai

Can LLMs invent better ways to train LLMs?

Scaling Synthetic Data Creation with 1,000,000,000 Personas



Higher capability and emergent skills



Software tuning accelerates evolving workloads

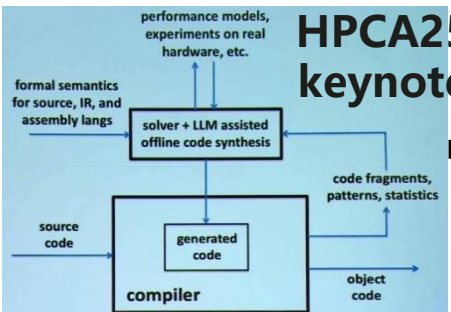
AI generates over 25% of Google's new code



Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling

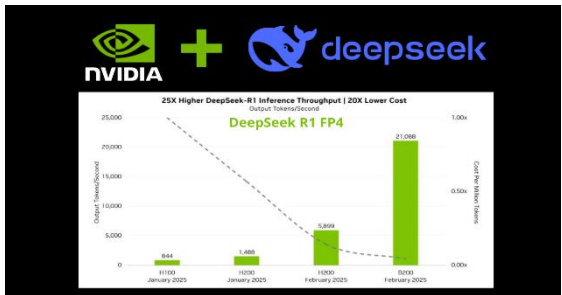
Feb 12, 2025
By Terry Chen, Bing Xu and Kirithi Devlekar

+68 Like Discuss (2)



HPCA25 keynote

Efficient kernels and inference software improve ROI



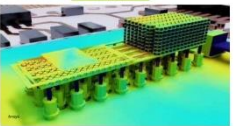
Shorter hardware development cycles

Faster chip simulation, factory optimization, lithography, computational lithography and design assistance

NVIDIA
ming Pro Graphics Robotics
Why 3D Visualization Holds Key to Future Chip Designs

At the Design Automation Conference this week, Ansys explores how NVIDIA Omniverse and Modulus help accelerate simulation workflows.

April 24, 2024 by David Rodriguez



NVIDIA
ming Pro Graphics Robotics
Staying in Sync: NVIDIA Combines Digital Twins With Real-Time AI for Industrial Automation

NVIDIA software — Omniverse, Metropolis, Isaac and cuOpt — combine to create an AI gym where robots, AI agents can work out and be evaluated in complex industrial spaces.



NVIDIA cuLitho

Accelerate computational lithography

NVIDIA cuLitho is a library with optimized tools and algorithms for GPU-accelerating computational lithography and the manufacturing process of semiconductors by orders of magnitude over current CPU-based methods.

Manufacturing computer chips requires a critical step called computational lithography — a complex computation — involving electromagnetic physics, photochemistry, computational geometry, iterative optimization, and distributed computing.

This computational lithography step is already one of the largest compute workloads in semiconductor production, necessitating massive data centers, and the silicon miniaturization evolution process exponentially amplifies the computation requirements over time.

ChipNeMo: Domain-Adapted LLMs for Chip Design

Model for "The Best, Smallest, Cheapest" Neural Network Design for AI Accelerators. This model is designed to assist in the design of neural networks for AI accelerators, focusing on minimizing size, maximizing performance, and reducing cost. It is trained on a large dataset of neural network designs and is able to generate designs that are optimized for these goals.

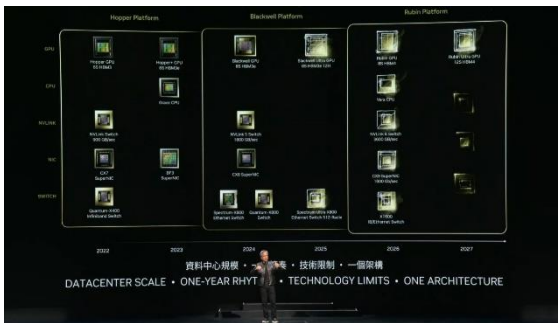
ChipNeMo is a domain-adapted LLM for chip design. It is trained on a large dataset of chip designs and is able to generate designs that are optimized for specific goals. It is designed to assist in the design of neural networks for AI accelerators, focusing on minimizing size, maximizing performance, and reducing cost. It is trained on a large dataset of neural network designs and is able to generate designs that are optimized for these goals.

Robotic Factories Supercharge Industrial Digitalization as Electronic Makers Adopt NVIDIA AI and Omniverse

NVIDIA Omniverse, Isaac and Metropolis Enable Delta Electronics, Foxconn, Pegatron, Wistron to Digitally Build, Simulate and Operate Factory Digital Twins

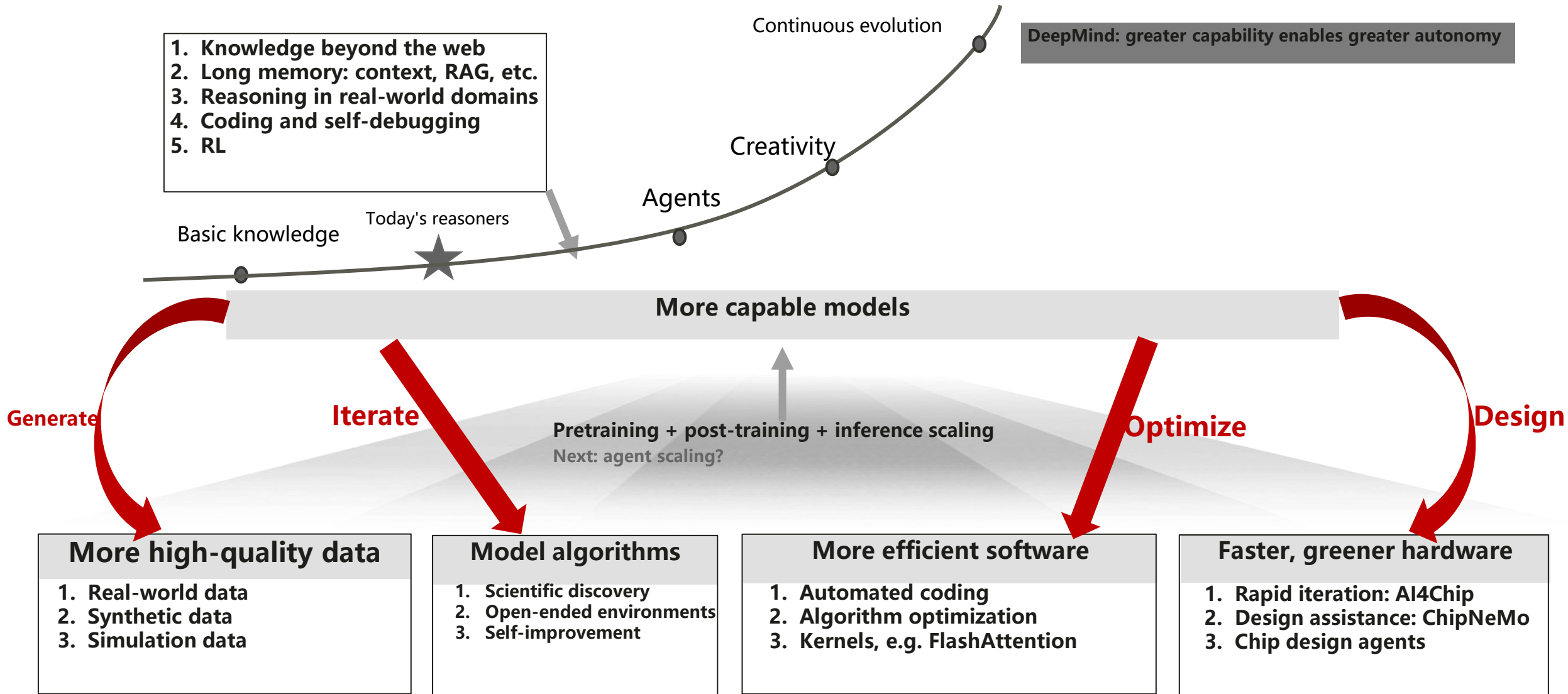
June 2, 2024

2-year generations, annual releases



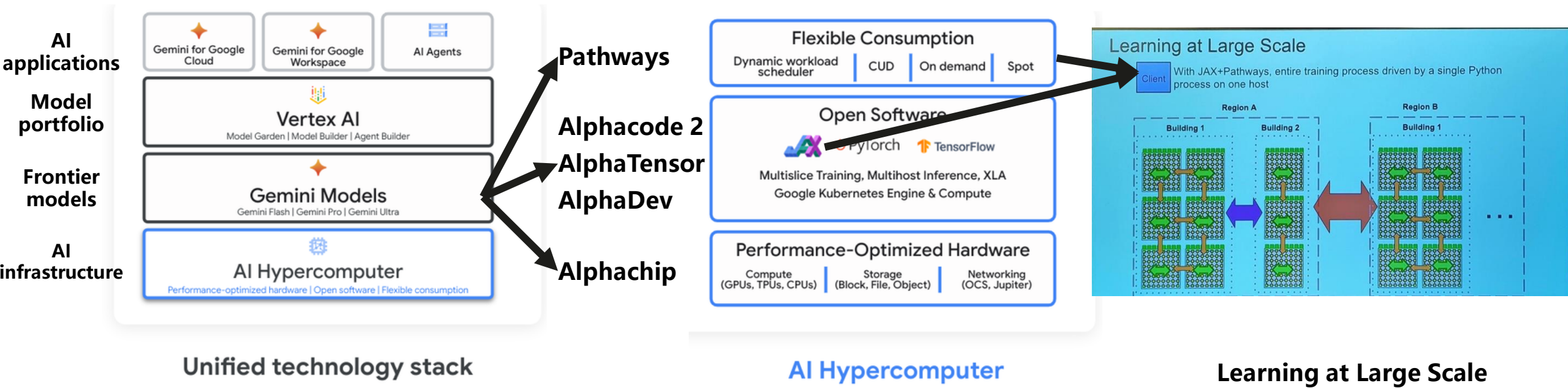
Agents: the next stage of model development

Better data, evolving algorithms, efficient software and new hardware



AI accelerates the software stack
Automated coding and kernel optimization

Google: a unified Gemini stack supports AI Hypercomputer and frontier model development



Learning for systems: expand ML across operating systems, compilers and memory management. This shift is already beginning.

Jeff Dean, MLSys 2024

Learning for Systems

Traditional systems (OS, compilers, memory allocators, storage systems, ...) do not make extensive use of machine learning today

This should change (and is starting to change)!

Self-optimizing code and algorithms enable agent self-improvement

Coding boosts productivity and developer adoption

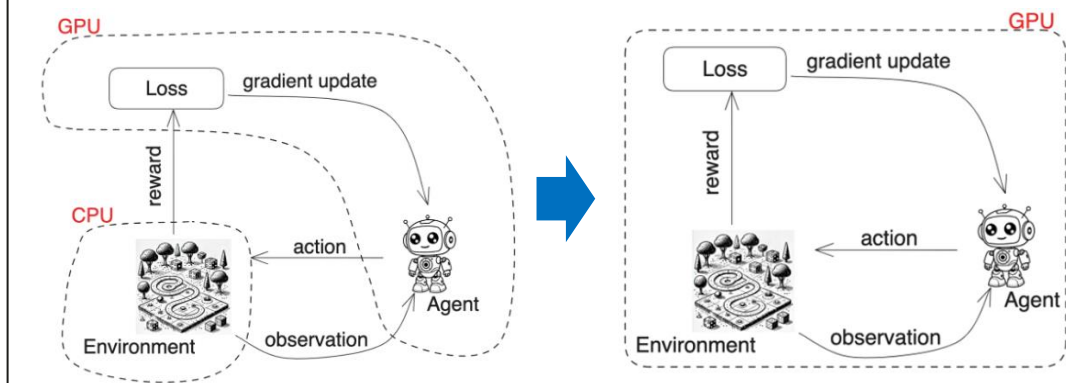
Example 1: Gemini Code Assist, powered by Gemini 2.0 (25 Feb 2025)

Product integration and self-debugging without human feedback

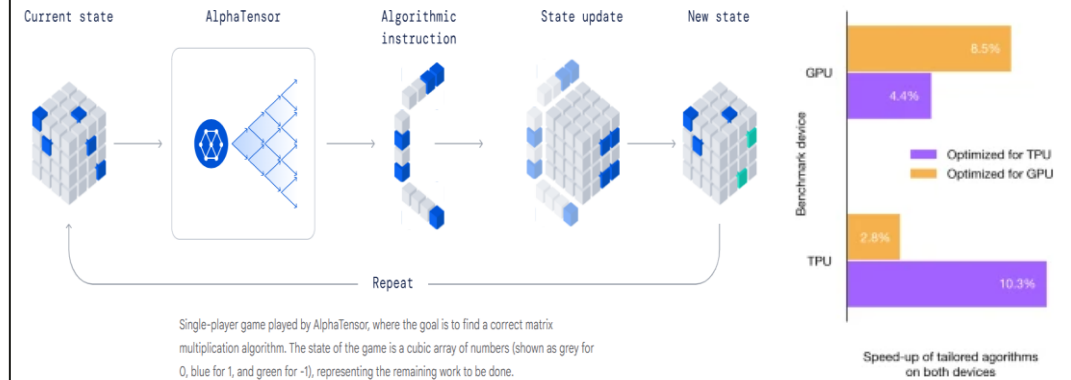
Gemini Code Assist uses large language models (LLMs) from Google. The LLMs are fine-tuned with billions of lines of open source code, security data, and Google Cloud documentation and sample code. These models paired with Gemini Code Assist give developers code completion, code generation, natural language chat, and more, in their IDE, and Google Cloud services including Firebase, Colab Enterprise (Vertex AI), Databases BigQuery, Apigee, and Application Integration.

Example 2: LLM coding accelerates RL (Chris Lu, Jacob Foerster)

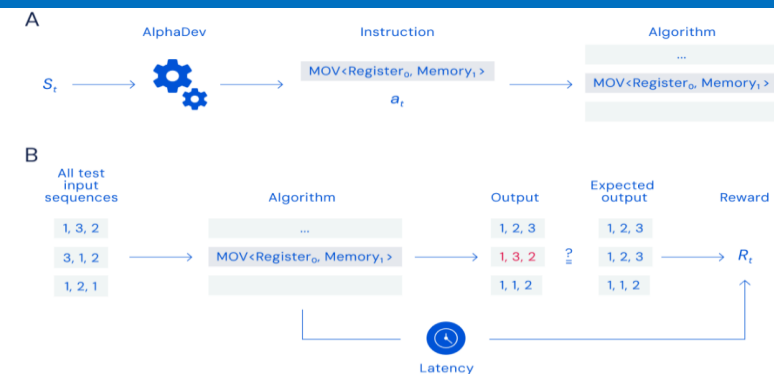
- Moving simulation from CPU to GPU speeds up RL by 1,000x.
- JAX enables large-scale GPU deployment, autodiff and parallelism.
- LLMs migrate code and generate or optimize RL algorithms.



AlphaTensor: faster matrix multiplication on TPUs and GPUs

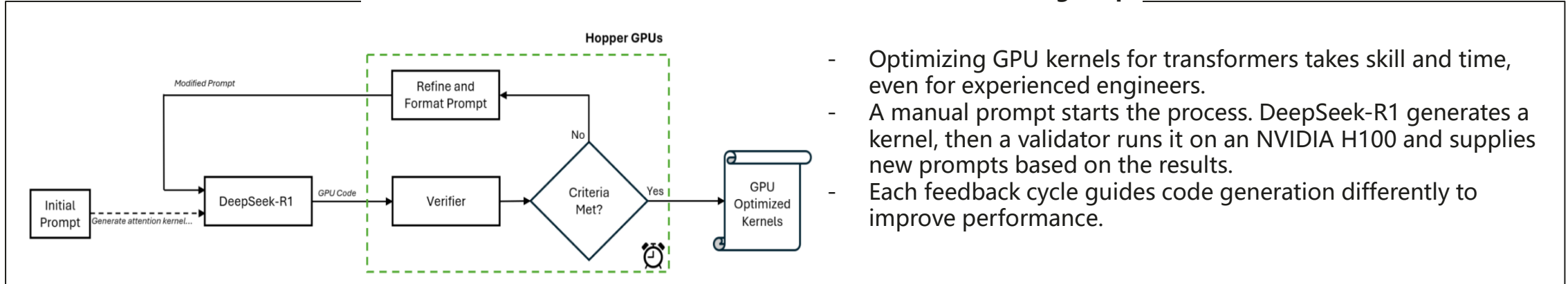


AlphaDev: faster sorting for data processing

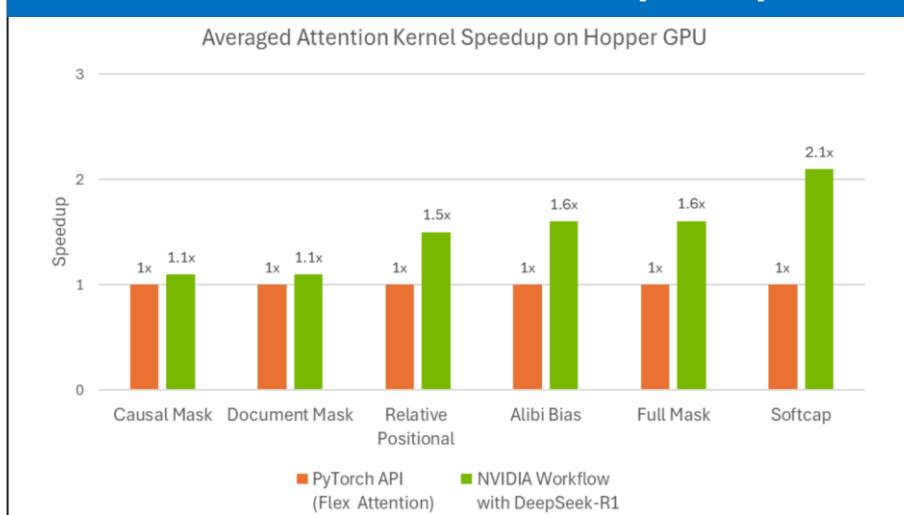


NVIDIA: DeepSeek-R1 automatically optimizes GPU kernels

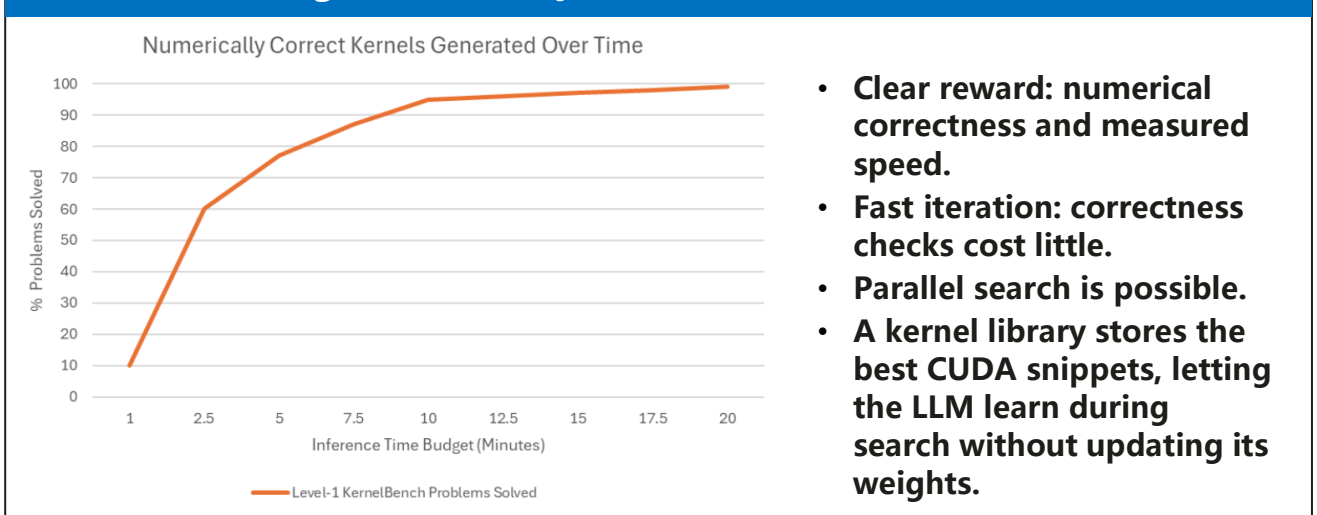
Generate, run and validate kernels in a 15-minute reasoning loop



Generated kernels deliver H100 speedups



Test-time scaling: numerically correct code in 20 minutes



Implications

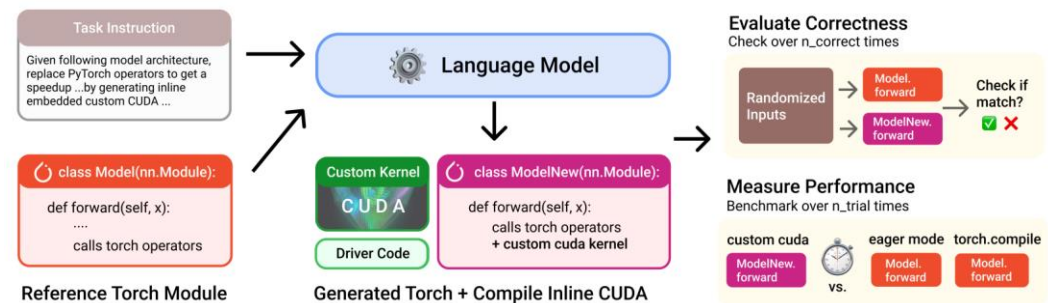
Models and test-time compute (TTC) are improving, while many kernels remain unoptimized.

More efficient kernels can speed up inference, enabling further TTC scaling and another round of kernel improvements.

KernelBench: LLM-generated kernels for faster, cheaper, greener AI

Kernel tuning takes expert time; LLMs can automate it

Framework for kernel generation and evaluation



- Kernel engineers use compiler feedback, profiling, hardware specifications, instruction sets and optimization expertise. Tools include PTX, ThunderKittens and Triton.
- **The framework automates engineering workflows across AI algorithms, languages and hardware. It checks speed and correctness programmatically and captures execution and profiling feedback.**
- **The LM decides which PyTorch code to optimize and how, within an automated execution loop.**

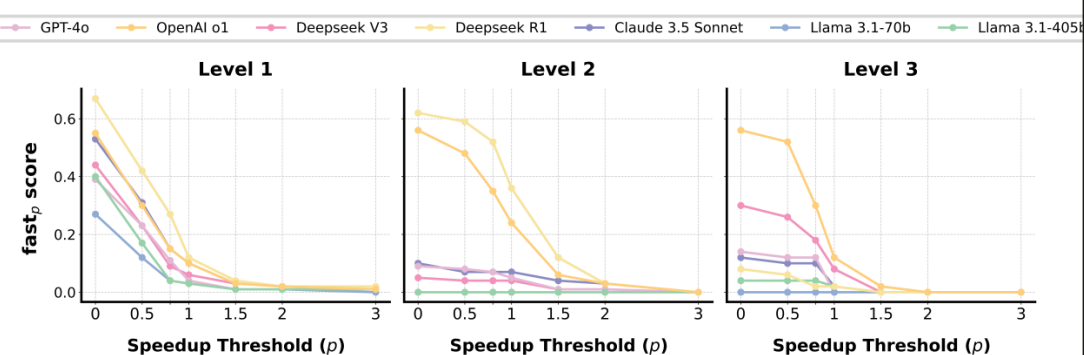
Tests optimization across three AI workload levels: single operations, sequences of operations and end-to-end architectures.

KernelBench tests LM optimizations on three levels of AI workloads:

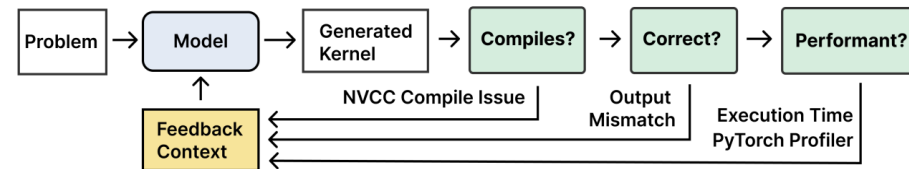
1. **Individual operations:** We include various AI operators, including matrix multiplies, convolutions, activations, norms, and losses. While PyTorch already uses expert-optimized closed-source kernels, making this a potentially challenging baseline, it is valuable if LMs can generate open-source kernels for the operations.
2. **Sequence of operations:** We provide problems that contain 3-6 individual operations together (e.g. a mainloop operator like matmul followed by pointwise operators like ReLU and Bias). This enables evaluating the models' ability to fuse multiple operators.
3. **End-to-end architectures:** We select architectures from popular AI repositories on Github including pytorch, huggingface/transformers, and huggingface/pytorch-image-models. These architectures contain many operations.

LLMs can accelerate kernels, with much room to improve

Non-reasoning LMs often produce incorrect or slower kernels



Rich feedback is key to better performance



- **R1 > V3 > Llama 3.1**
- **Repeated sampling improves test-time scaling.**
- **Feedback on code quality, execution and profiling can substantially improve performance.**

Method	Level 1			Level 2			Level 3		
	Llama-3.1 70B	DeepSeek V3	Deepseek R1	Llama-3.1 70B	DeepSeek V3	Deepseek R1	Llama-3.1 70B	Deepseek V3	Deepseek R1
Single Attempt (Baseline)	3%	6%	12%	0%	4%	36%	0%	8%	2%
Repeated Sampling (@10)	5%	11%	N/A	3%	14%	N/A	1%	14%	N/A
Iterative Refinement w G	9%	9%	18%	0%	7%	44%	0%	14%	4%
Iterative Refinement w G+E	5%	13%	41%	5%	62%	62%	8%	22%	12%
Iterative Refinement w G+E+P	7%	19%	43%	4%	6%	72%	2%	14%	18%

METR: evaluating optimized kernels to accelerate the ecosystem

KernelBench limitations

- **Tests are too lenient: outputs need only match a reference within 0.01 absolute tolerance. Real workloads also require model quality, training stability or stricter scalar tolerances, making k-best selection harder.**
- **The baseline is unoptimized PyTorch, not tuned CUDA. This suits small research teams but does not represent large-scale production.**
- **No human-expert comparison: scores are hard to interpret without experts solving the same tasks.**
- **Single-GPU tests omit the inter-GPU communication that frontier workloads require.**



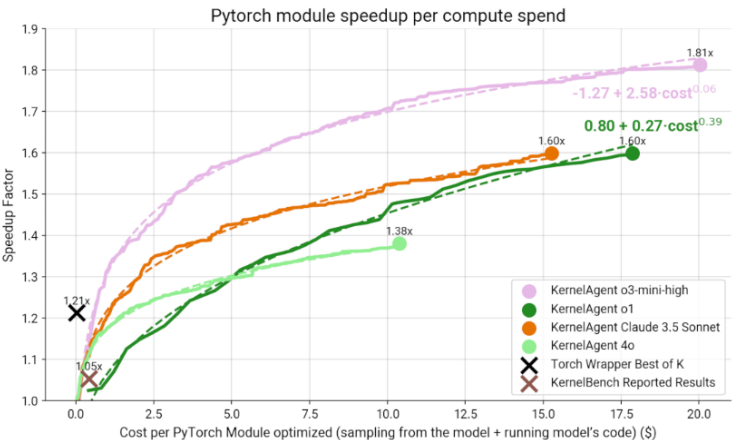
METR: use LRMs to tune frequently used kernels

- Four levels: complexity, kernel count and expert time.
- Single operations, operation groups, full models and SOTA libraries.

Level	# problems	Description	Realism	# Kernels per problem	Expert Time Estimate
1	90	Single PyTorch operations, eg CrossEntropyLoss	Realistic, memorizable	1	15 min - 4 hours
2	80	Sequences of 3-6 PyTorch operations, eg Linear->MaxPool3d->ReLU	Unrealistic, novel	1-3	30 min - 10 hours
3	37	Whole architectures from 2010s, eg AlexNet, GRU	Realistic, memorizable	10+	8-100+ hours
5	14	Frontier of open source capabilities and complexity in 2024	Realistic	10+	40-500+ hours

- Optimizing KernelBench tasks with o3-mini breaks even after 27 hours of H100 compute on average.
- A PyTorch module used for more than 27 hours may therefore cost less to run after optimization.

Inference scaling helps; o3-mini-high leads



The importance of appropriate elicitation and compute spend: performance increases dramatically with better scaffolding and more samples. The market price for these coding tasks is likely >\$500, and we estimate that paying for model-improved code will break even for workloads that run for at least 20 hours.

- **Collaborators: Nebius, Modal Labs, Prime Intellect and Together AI.**
- **Useful techniques: operator fusion and better shared-memory and register use.**
- **Next steps: specialized fine-tuning, stronger reasoning, agent workflows, better datasets and other hardware platforms.**

Expert view: current agents cannot yet improve frontier kernels.

- Private discussions with frontier labs indicate substantial spending and engineering time on inference kernels.
- Agent-written kernels match or underperform top experts, limiting their immediate value to these labs.

Sakana AI: automated model development, training methods and kernels

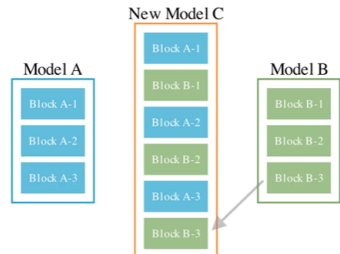
Implications

1. AI explores vast search spaces using coding, reasoning and test-time compute.
2. Further progress needs longer context for profiling and errors, stronger benchmarks, and high-quality task-specific reasoning data.

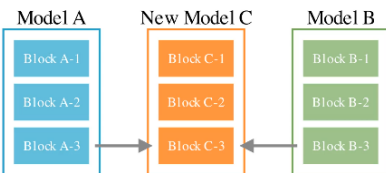
Evolutionary Model Merging: search for the best model combinations

- Low cost: no pretraining required
- Combine models to suit downstream tasks

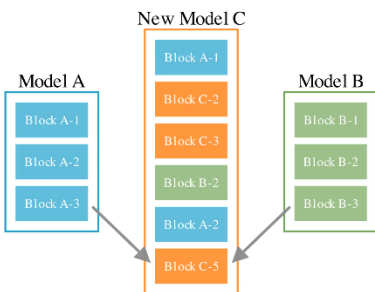
Layer merging



Parameter merging



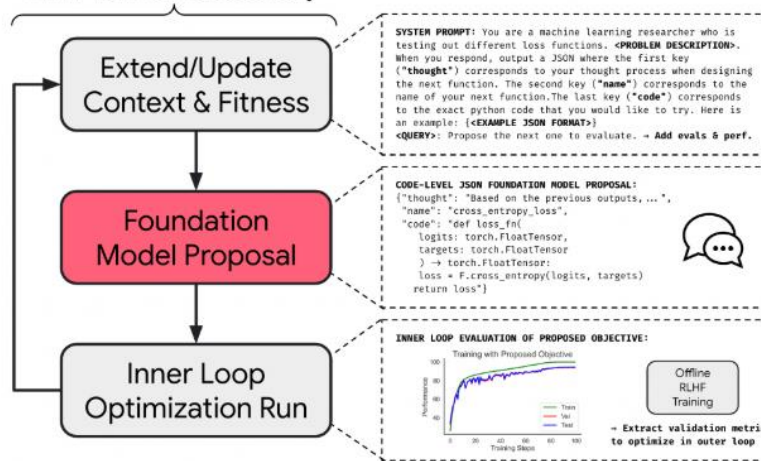
Combined merging



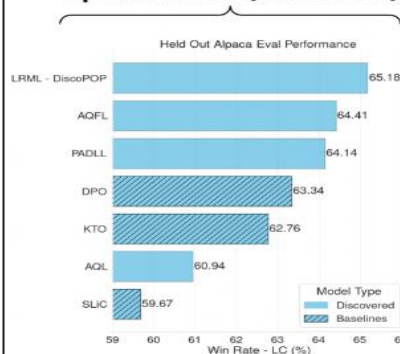
LLM-squared: iteratively improve preference losses

Update context, generate a loss function, test it with RLHF, then feed results back into the next round.

LLM-Driven Discovery



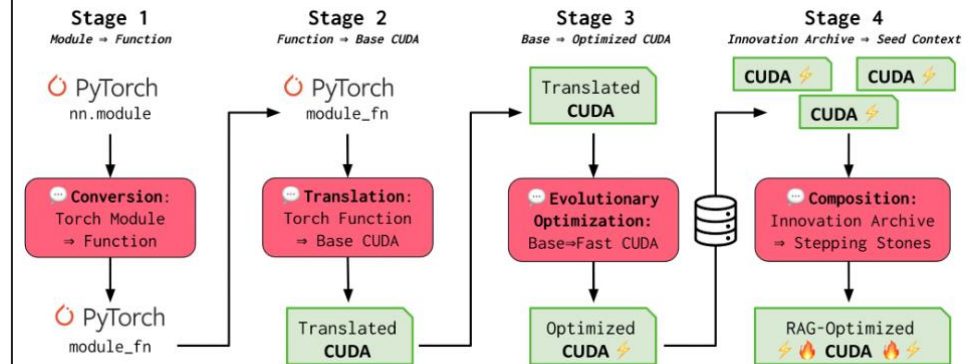
Discovered Preference Optimization (DiscoPOP)



- Found losses that outperform DPO.
- Uses information from each generation.
- More TTC can further improve results.
- May extend to other training and inference components.
- Depends on base-model coding and reasoning.

CUDA engineer: mean 1.34x speedup on 250 tasks

- Convert PyTorch modules into functions, then baseline CUDA kernels.
- The LLM iteratively improves these kernels.
- Released 30,000 agent-generated CUDA kernels.



Level 2 [Fused Ops]	# Translated Operations	Improved over torch	Improved over compile	Improved over KernelBench	Torch P50 Speedup	Torch P75 Speedup	Torch Max Speedup
deepseek-v3 [pass@10]	39	14	7	9	0.49	1.08	8.14
sonnet3.5 [pass@10]	52	20	14	13	0.39	1.27	10.81
deepseek-R1 [pass@10]	70	23	16	17	0.37	1.06	59.19
o1-preview [pass@10]	95	42	13	22	1.00	1.01	3.35
o1-high [pass@10]	81	29	17	19	0.87	1.00	4.34
deepseek-v3 [loop@10]	43	15	10	12	0.96	1.11	3.35
sonnet3.5 [loop@10]	64	17	15	11	0.31	1.03	10.16
deepseek-R1 [loop@10]	80	21	18	14	0.26	1.00	112.45
o1-preview [loop@10]	97	45	19	26	1.00	1.04	3.38
o1-high [loop@10]	90	24	13	15	0.51	1.00	6.78
o3-mini-high [loop@10]	88	20	20	17	0.23	0.93	8.64
Union - Max All	98	72	40	41	1.05	1.31	112.45

- Validation flaws make reported speedups unreliable.
- **Promising method.**
- **Limited hardware insight.**
- **Some kernels slow down.**

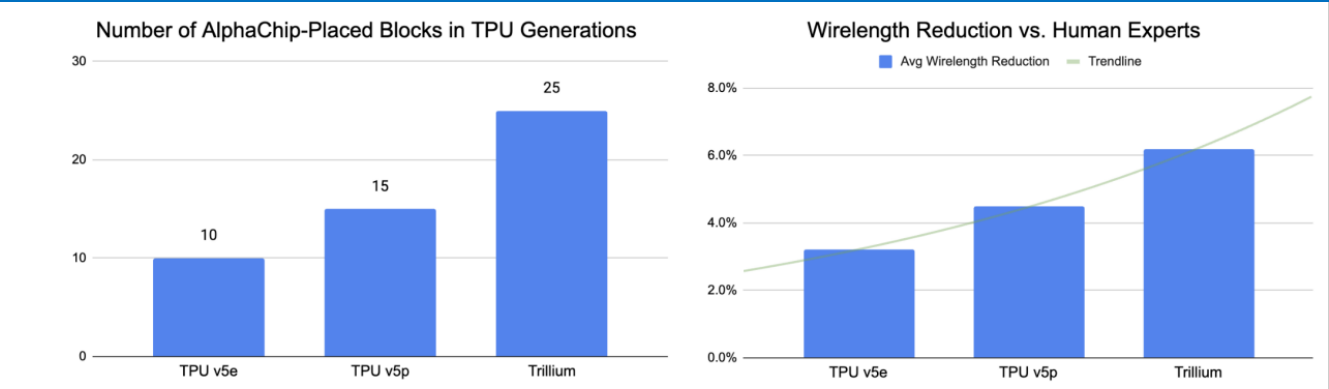
AI accelerates hardware design and optimization

Could new chip design cycles shrink to months?

Google: That chip has sailed. AlphaChip designs TPUs

Release gaps shrink from 3 years to 1: V3 (2018), V4 (2021), V5 (2023), V6 (2024). Forecast: V7 production in 2026.

AlphaChip spans 3+ TPU generations, covering more blocks and outperforming experts by wider margins

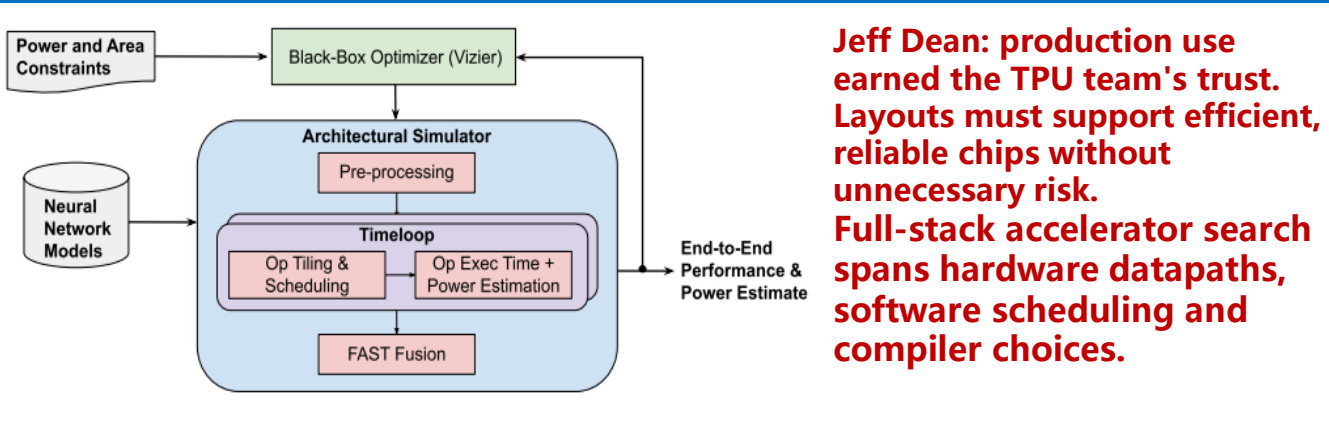


Mar 2024: 7 AlphaChip layouts adopted in Google Axion Processors (ARM-based CPU).

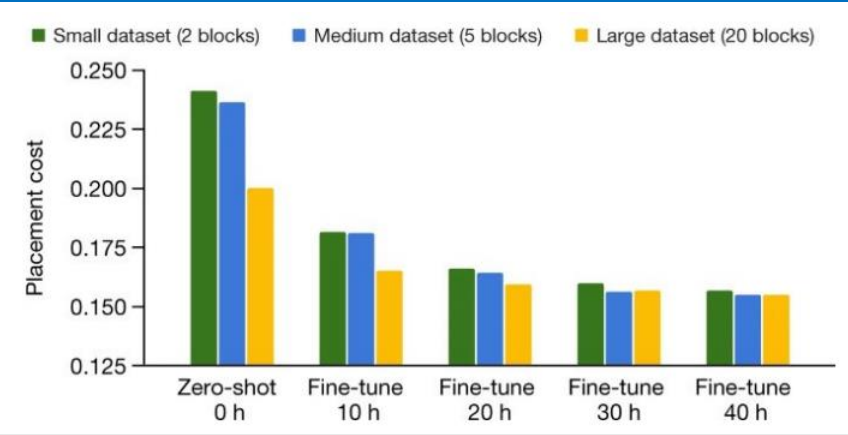
Fast-changing workloads demand faster chip iteration

	Workload share
TPU v1	MLP/DLRM: 61% RNN: 29%
TPU v3	MLP/DLRM: 27% RNN: 21%; CNN: 24% Transformer: 21%
TPU v4	MLP/DLRM: 24% CNN: 12% Transformer: 57%
TPU v5/v6	transformer

AlphaChip and full-stack RL search: average 3.7x Perf/TDP



More pretraining and SFT improve results



NVIDIA: AI designs chips to train better AI

Better chips through broader AI and LLM adoption

AI for Chip Design Research @ NVIDIA

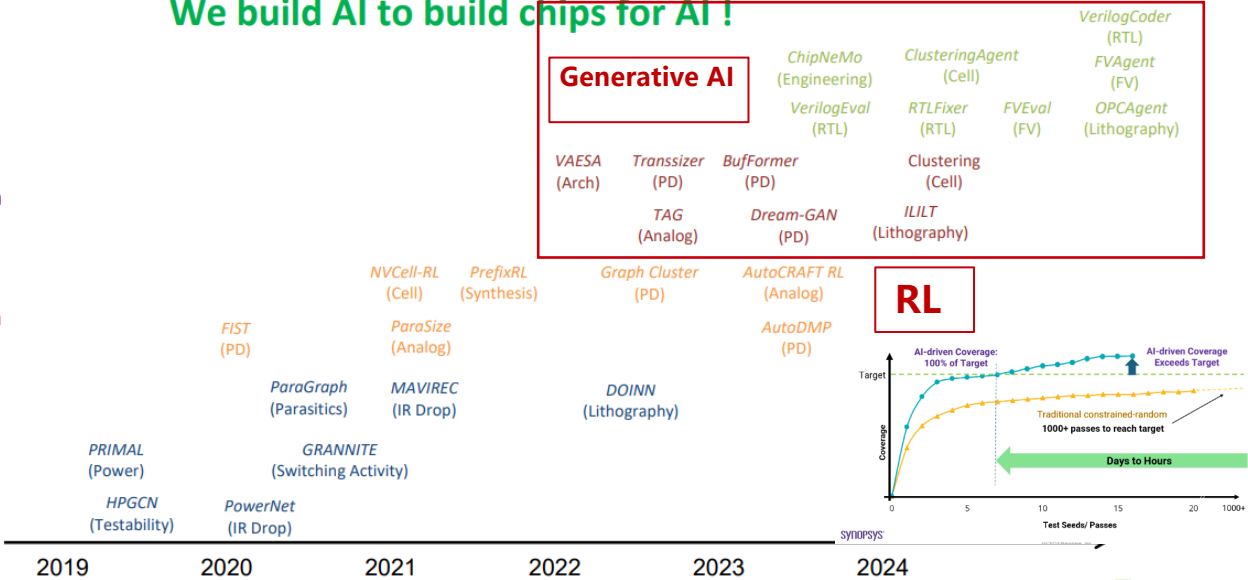
We build AI to build chips for AI!

Design Assistance

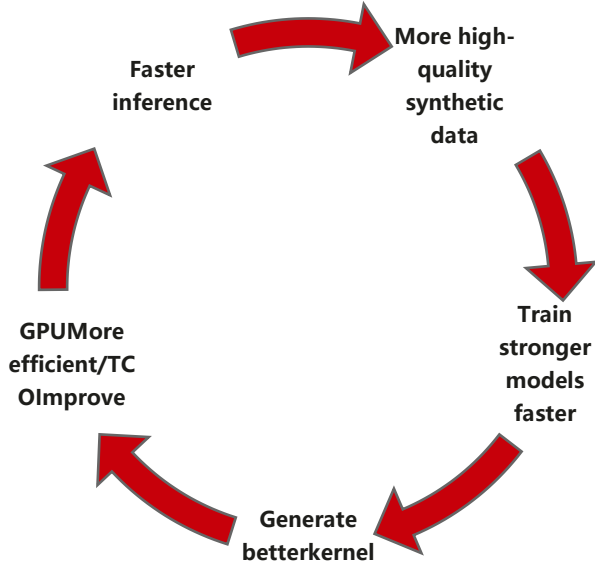
Design Optimization (Gen AI)

Design Optimization (RL, BO)

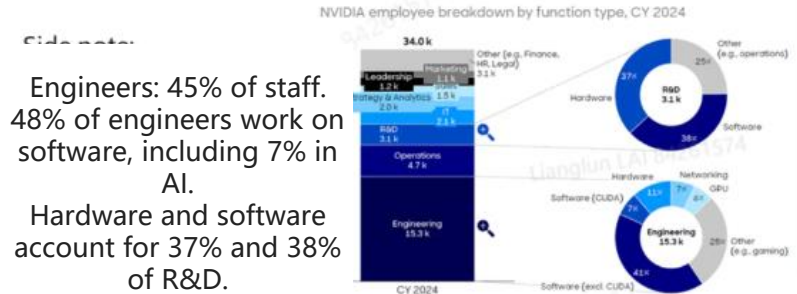
Design Analysis



Kernel optimization enables more inference scaling

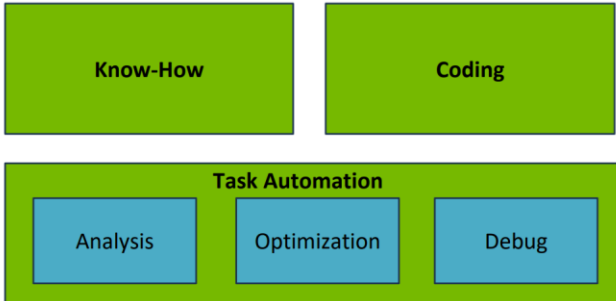


NVIDIA's software-heavy workforce benefits from AI coding and CUDA tuning



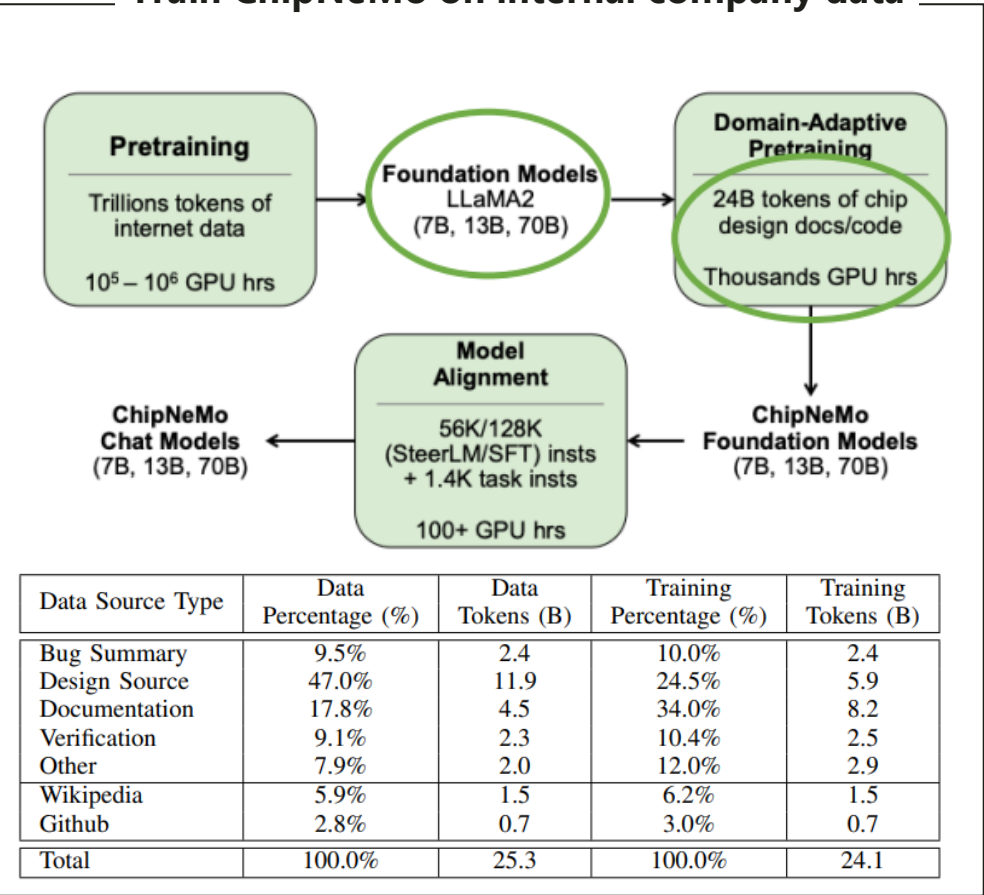
Chip engineers face many choices and constraints. AI explores trade-offs across logic, area, cell libraries, process parameters and EDA tools or configurations, combining vast design-space search with domain expertise.

LLM assistance: expertise, coding and automation



ChipNeMo: a foundation model for chip design tasks and engineer productivity

Train ChipNeMo on internal company data



A foundation for chip design assistants

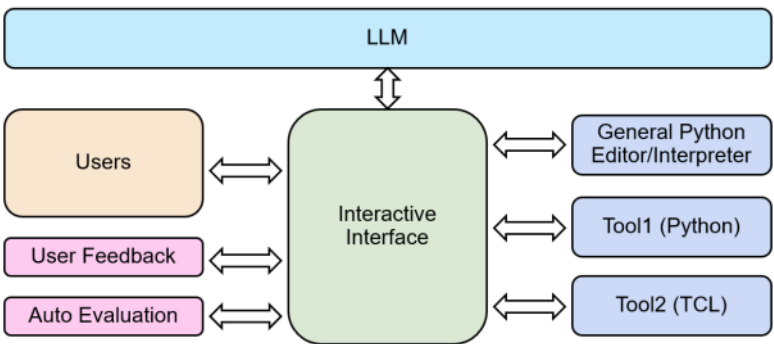
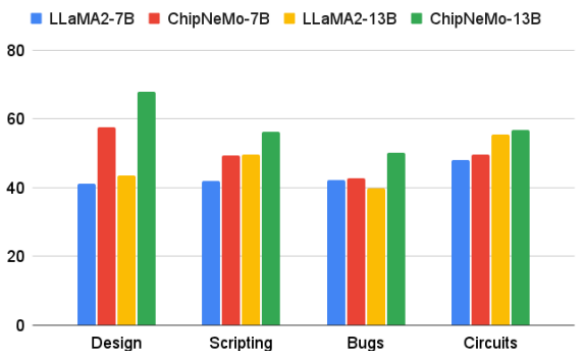


Fig. 4: LLM script generator integration with EDA tools



(a) Chip Design Domain Benchmarks.

NVIDIA engineers prioritize chatbots, code generators and analysis tools.

Automation: maintain updated descriptions of known bugs, saving time. The most popular use so far.

Chatbots: answer GPU architecture and design questions and locate technical documents.

Code generation: write in specialist chip design languages.

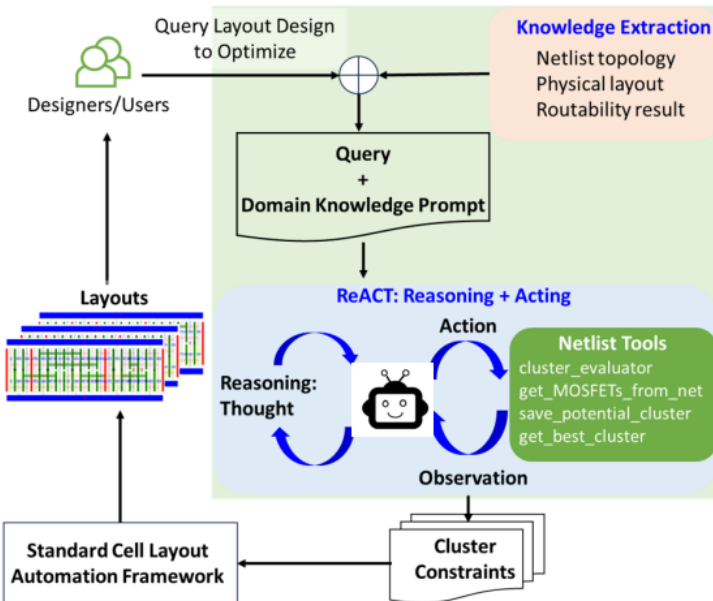
Insight: learn expert reasoning. Meeting transcripts provide CoT; summaries provide outputs.

- Haoxin Ren: GenAI could support every chip design stage, improving productivity through smaller, faster circuits and optimized large blocks.
- The foundation is clean training data: software, RTL, verification testbenches, hardware configurations and technical documentation.

Chip design agents combine LLM reasoning with domain expertise and tools to improve efficiency

LLMs reduce cell placement area by 5%

- LLM language and reasoning skills generate cluster constraints to optimize cell power, performance and area (PPA). ReAct helps debug routability.
- Tests on 2 nm sequential standard cells reduced cell area and repaired routing problems.

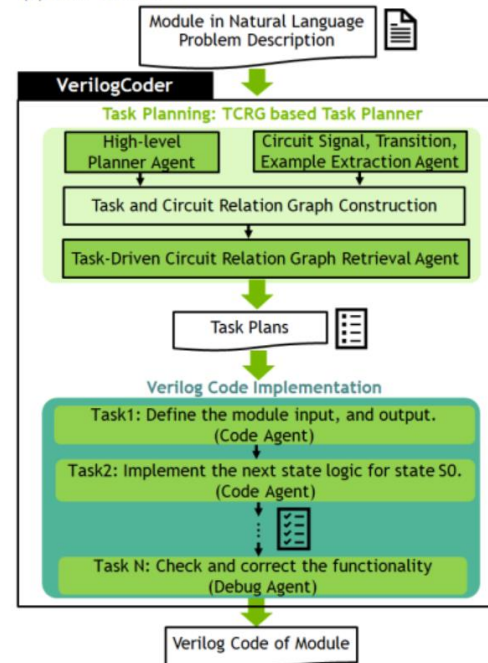


Build complex circuits from predesigned, verified logic cells.
C.-T. Ho et al., Large Language Model (LLM) for Standard Cell Layout Design Optimization

VerilogCoder: 94.2% correctness, 33.9% above SOTA

- AI agents generate Verilog code.
- They invoke syntax checkers, simulators and waveform tracers.
- Two stages: multiple LLM agents and retrieval plan tasks, then coding and debugging agents execute and verify subtasks.

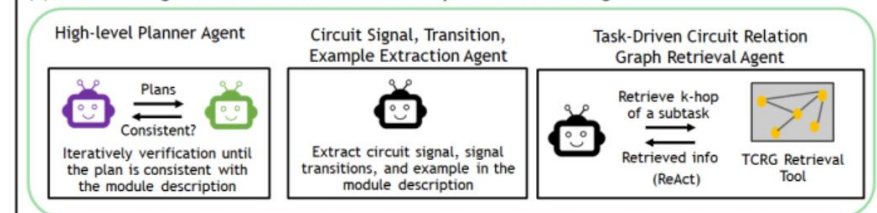
(a) Flow Overview



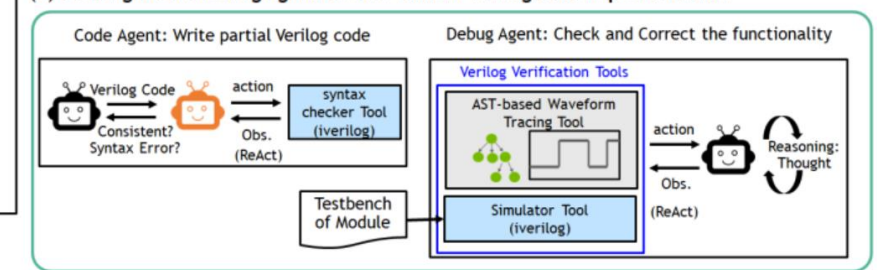
(b) Large Language Model (LLM) Roles of Multi-LLM Agents in VerilogCoder



(c) Multi-LLM Agents with Various Roles for Steps in Task Planning



(d) Code Agent and Debug Agent for Sub-Tasks in Verilog Code Implementation



Verilog HDL models digital circuits at algorithm, gate and switch levels.
C.-T. Ho et al., VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool

Autonomous model evolution

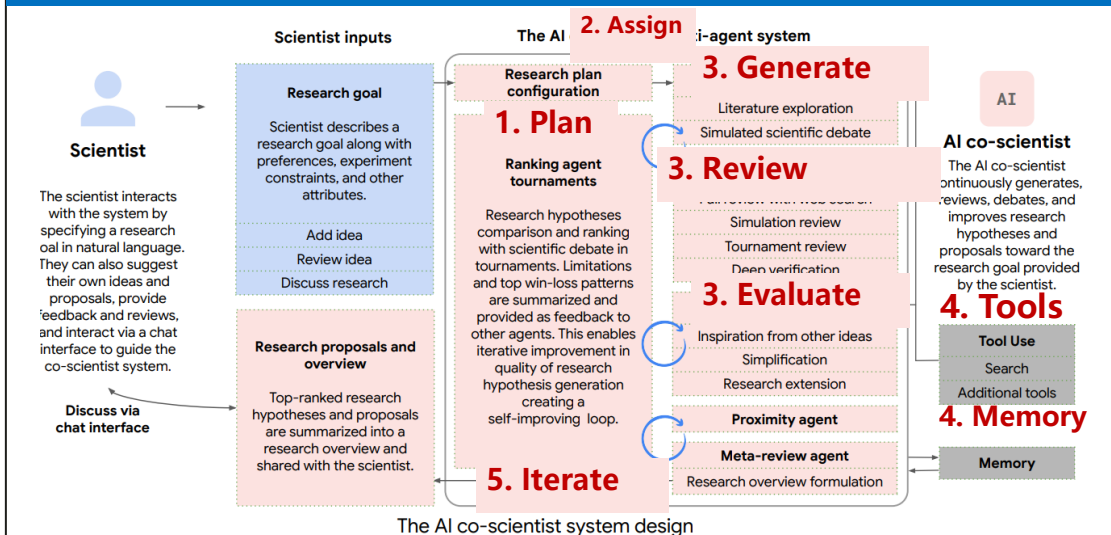
Environments, exploration and agent compute utilization

Google AI co-scientist: test-time scaling and self-play for scientific discovery

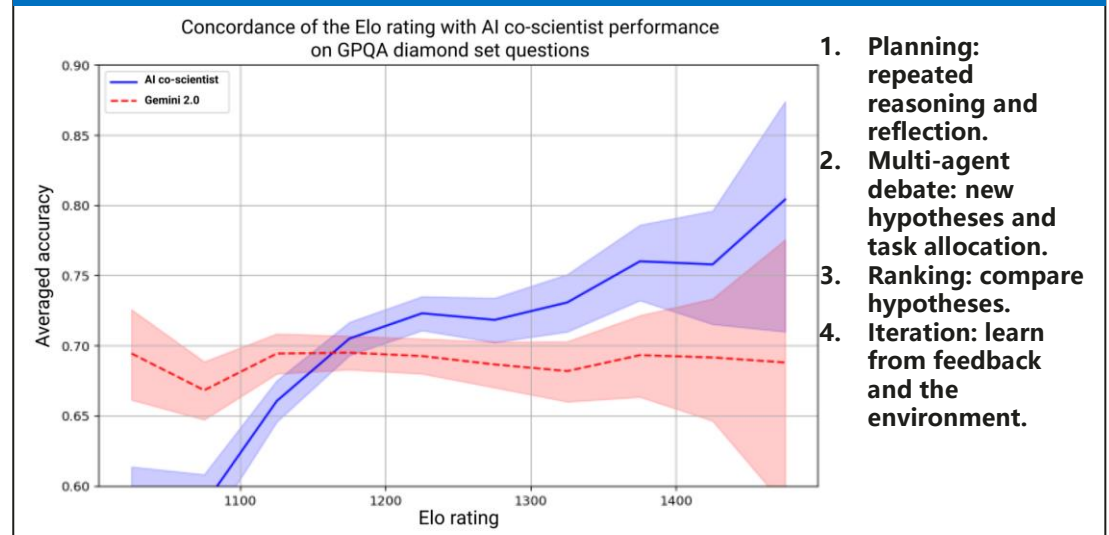
AI co-scientist uses Gemini 2.0, multiple agents and asynchronous task execution.

- **Asynchronous execution masks latency. Agents work continuously in a configurable framework. A supervisor manages queues, assigns subtasks and allocates resources for efficient compute use and iterative scientific reasoning.**
- Natural-language interface: researchers set goals, give feedback on hypotheses and solutions, and guide progress.
- Specialist agents: tailored prompts divide tasks using inductive biases and scientific priors.
- Persistent context memory stores and retrieves agent and system state, supporting extended computation and reasoning.

Agents generate, assess, debate and refine research hypotheses



TTC loops improve results through reasoning and reflection

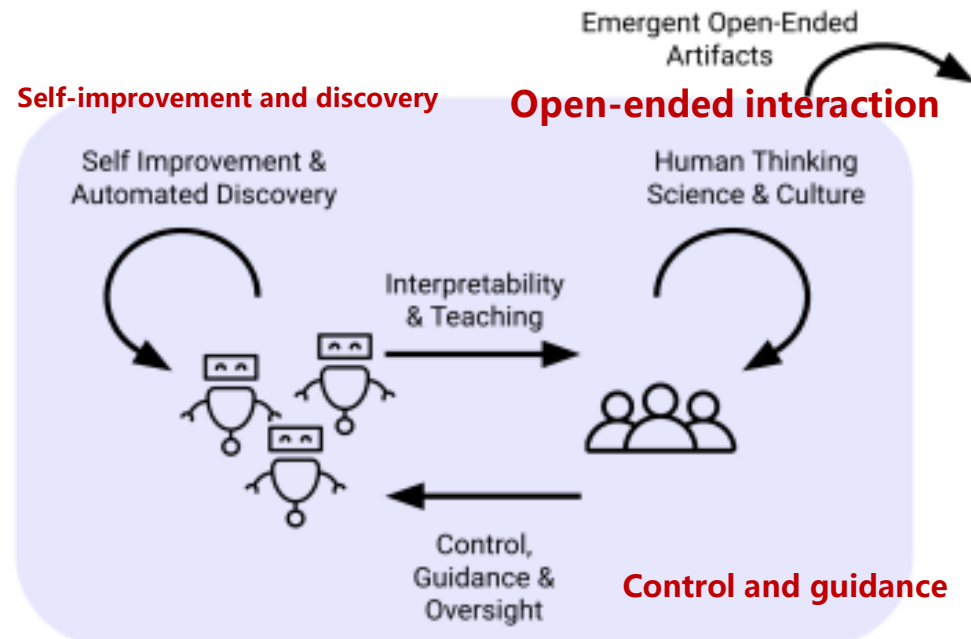


Cong Lu: agents can scale beyond human information limits, generate harder open-ended tasks, use long context to learn from failures, discover loss functions and collaborate in chains that combine knowledge and specialist capabilities.

GDM Open-Endedness, London: open environments are essential to ASI

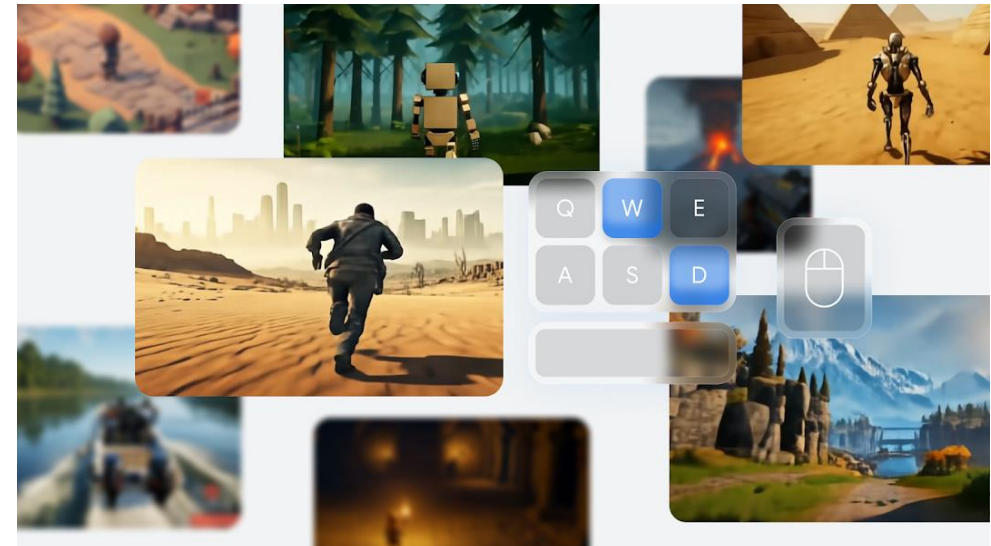
Open-ended environments enable self-improvement

- Real systems are difficult: how should goals, rewards and guidance granularity be defined?
- Open-ended means both novel and learnable.
- Training data must capture the abstractions needed for learning.
- Agents learn from video, audio, touch, sensors and actions. Modalities interact and reinforce one another, beyond simply providing multiple inputs.



Learning to create beyond solving set tasks

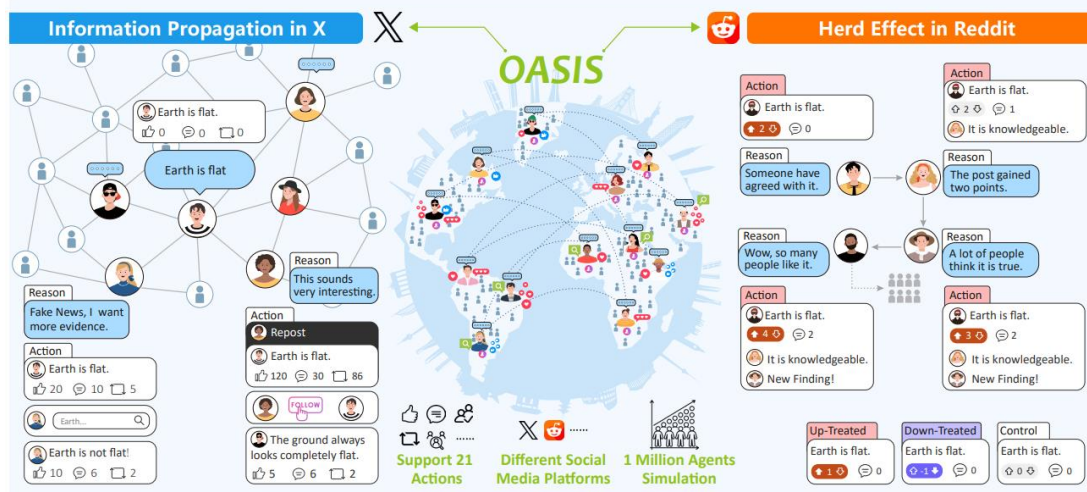
- Open-ended play develops general capabilities.
- **Games offer efficient reward design, but generalization and context remain challenges.**
- Agents learn latent relationships in unfamiliar environments, generate tasks, adapt and solve varied problems.
- They learn how to act and how actions change the world.
- **They create knowledge, hypotheses and insights beyond human training data.**
- Examples: Genie 2 and SIMA.



Games and agent collaboration create environments for learning and discovery

Oxford / CAMEL-AI OASIS: a million-agent simulation

- Agents post, comment and discuss online.
- As the population grows, interactions become more frequent and diverse. Collective behavior and feedback can produce emergent collective intelligence.



Microsoft Cambridge WHAM: actions and goals in 3D games

Observe the game, learn its rules and goals, then predict the next state



- 1 Observe the game environment
- 2 Input an action and predict the new state
- 3 Score player actions against the learned world rules

OpenAI's assessment: frontier model autonomy remains low

OpenAI tracks autonomy and reports evaluations in model system cards.

- Autonomy improves on long-horizon and agentic tasks.
- Models show potential for self-improvement and AI research, but still struggle with open-ended ML research.
- **Autonomous Replication and Adaptation (ARA): gaining capability by acquiring more real-world resources.**
- Models struggle to acquire and use resources effectively, including low GPU utilization.

Implications

- Deep Research is an agent product that combines knowledge and reasoning.
- This combination could enable creative discovery.
- It may lay the groundwork for future innovator agents.
- Innovation could, in turn, improve reasoning and agent capabilities.

Benchmarks and capabilities

Short ML tasks

Evaluation	Capability
OpenAI Research Engineer Interview: Multiple Choice and Coding	Basic short horizon ML expertise

Real software engineering

SWE-bench Verified	Real-world software engineering tasks
--------------------	---------------------------------------

Resource acquisition

Agentic Tasks	Basic software engineering tasks related to fraud and resource acquisition
---------------	--

Data and ML tasks

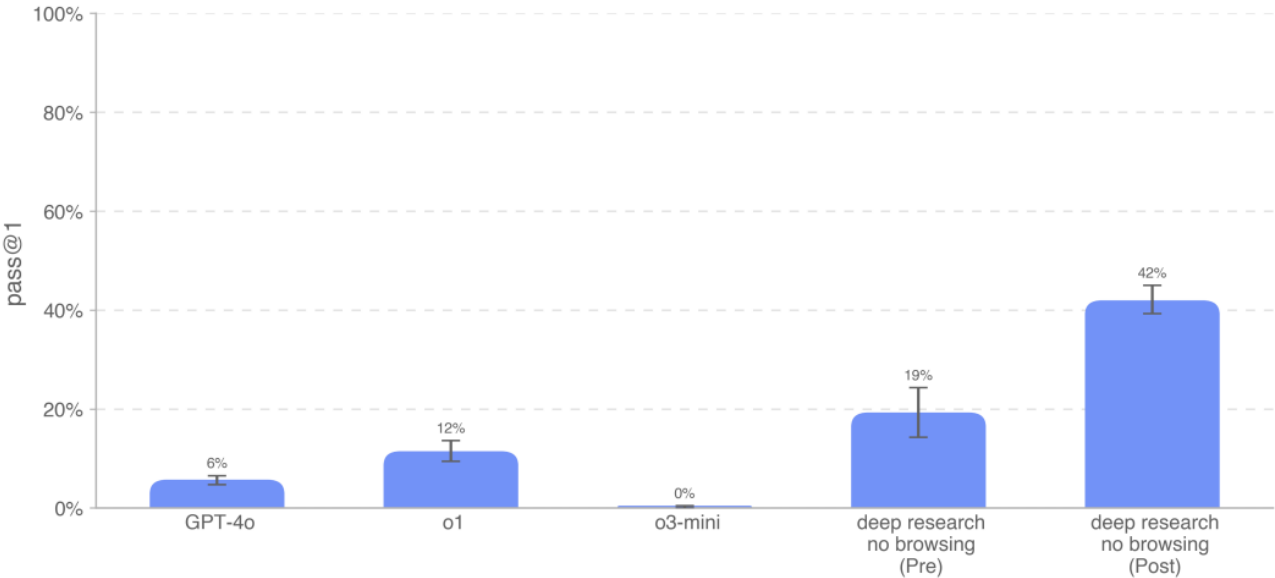
MLE-Bench	Real world data science and ML competitions
-----------	---

ML research

OpenAI PRs	Real world ML research tasks
SWE-Lancer	Real world software engineering tasks

o-series models struggle with ML research; Deep Research improves results

OpenAI PRs



New hardware–software co-design for reasoning agents: higher AFU?

OpenAI agents underuse available resources

1. Plans use CPUs for small models and GPUs for large ones.
2. None of OpenAI's three tested agent strategies uses compute and time efficiently.
3. Memory or storage limits sometimes end tasks early.

An extra GPU does not improve results in the same time

Table 3: Comparing the performance of GPT-4o (AIDE) on different hardware configurations (averaged over 3 seeds, except for *Standard* which used 36 seeds).

Setup	Description	Achieved Medal (%)
CPU-only	Same as <i>Standard</i> but with no GPU provided.	9.1 ± 1.0
Standard	36 vCPUs with 440GB RAM and one 24GB A10 GPU.	8.7 ± 0.5
Extra GPU	Same as <i>Standard</i> but with two 24GB A10 GPUs.	10.2 ± 2.0

Agents rarely estimate their code's runtime

1. Agents cannot estimate time and compute per pass.
2. They do not understand available system capacity.
3. They therefore fail to use all compute resources.

Possible cause 1: training lacks reasoning about compute use, code efficiency and resource requirements.

Possible cause 2: current inference systems rely on developers to configure local optimizations.

Why it matters: agents accelerate model development

- From coding agents to model research agents
- OpenAI tests: answer interview questions, implement model-development code, and complete model-development tasks.

High model utilization does not ensure high agent utilization

- Agent resource use, with or without human help:
- **Agent FLOPs Utilisation (AFU)?**
- **Agent Bandwidth Utilisation (ABU)?**
- **Agent Memory Utilisation (AMU)?**
- Which mix of heterogeneous or distributed compute?

Inference systems for agents

Models and agent inference frameworks need to:

1. Recognize available compute resources.
2. Estimate task time and compute costs.
3. Track utilization and plan resource use.
4. Schedule resources to optimize themselves.

Insight: expert reasoning as high-quality data

- Meeting transcripts as CoT.
- Meeting summaries as outputs.
- Models learn how human experts reason.

Implications

- **Develop frontier models and use them broadly in daily work to accelerate compute infrastructure.**
- **Build stronger inference systems that generate tokens faster and more cheaply. Annual token output could become a core competitive advantage.**
- Improve model coding skills, including kernel optimization that accelerates hardware.
- **Strengthen autonomous scientific discovery with expert reasoning data: transcripts as CoT, and meeting topics and summaries as question–answer pairs.**
- **Design hardware and software for AI as a workforce, progressing beyond productivity assistance and continually adopting stronger models.**